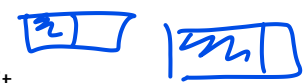
When the model says 0.9, is it bad 90% of the time?

Last lecture measured how well our cheese detector *ranks* batches (ROC, PR). But it also outputs a **number** — $\hat{p} = 0.9$. If we tell the factory "90% chance this batch is bad," is that *true*?

A model can **rank** perfectly (high AUC) and still output probabilities that are systematically wrong — saying 0.9 when only 70% of such batches are actually bad. That gap is **(mis)calibration**.

This deck: what calibration is, how to measure it, and how to fix it.



Outline

What calibration is

Why it matters

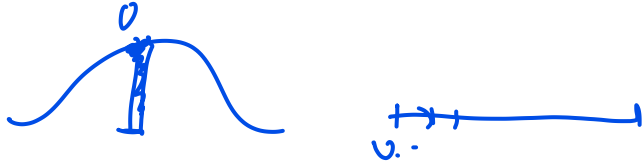
Fixing it

What "calibrated" means

A classifier is **calibrated** if its probabilities match reality:

> Among all batches it scores $\hat{p} \approx 0.8$, about **80%** are actually bad.

- A **score** ranks; a **calibrated probability** can be read as a real chance.
- Calibration is about the *value* of $\hat{p}$, not the *order* — a separate property from accuracy or AUC.
- We check it by bucketing predictions and comparing predicted vs. observed frequency.



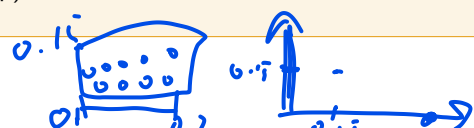
Building the reliability diagram

Take the model's predictions on held-out data and:

1. **Bin** the predicted probabilities $\hat{p}$ — say 10 equal-width bins $[0, 0.1), [0.1, 0.2), \ldots, [0.9, 1]$.
2. In each bin compute two numbers: the **mean predicted** $\hat{p}$ ($\rightarrow x$) and the **observed fraction of positives** ($\rightarrow y$). *Of the batches this bin scored "≈ 0.8", how many were actually bad?*
3. **Plot** each bin as a point (x, y) and join them.

A perfectly calibrated model has $y = x$ in every bin — the points land on the **45° diagonal**.

Not a CDF. Both axes are probabilities and it rises left-to-right, so it *looks* like one — but it is **not cumulative**. Each point is a *local* average inside one bin, and the reference is the diagonal $y = x$ (not a CDF's $0 \rightarrow 1$ ramp). The same model can sit *above* the diagonal in some bins and *below* it in others.



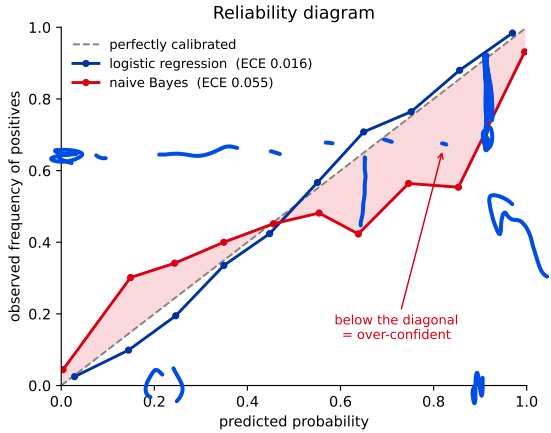
Reading the reliability diagram

Compare each bin's point to the diagonal:

- **on** the diagonal = perfectly calibrated;
- **below** = **over-confident** (predicted higher than reality);
- **above** = **under-confident** (predicted lower).

Logistic regression tracks the diagonal; naive Bayes bows away from it, pushing $\hat{p}$ toward 0 and 1.

Its **ECE** (*expected calibration error*) — the bin-size-weighted average of those vertical gaps — is 0.055, vs. 0.016 for the calibrated model. We compute one by hand next.



Worked example: computing ECE by hand

Bin the predictions; in each bin compare the **average prediction** $\hat{p}_b$ to the **observed frequency** of bad batches, then average the gaps *weighted by bin size*.

bin	n_b	$\hat{p}_b$ / obs.	$\lvert\text{gap}\rvert$
$[0.0, 0.2)$	50	0.10/0.10	0.00
$[0.2, 0.4)$	20	0.30/0.25	0.05
$[0.6, 0.8)$	20	0.70/0.50	0.20
$[0.8, 1.0]$	10	0.90/0.60	0.30

$n = 100$ predictions; empty bins contribute 0.

Weight each bin's gap by its share of all predictions:

$$\text{ECE} = \sum_b \frac{n_b}{n} \left|\text{obs}_b - \hat{p}_b\right|$$

$$= \tfrac{50}{100}(0) + \tfrac{20}{100}(.05) + \tfrac{20}{100}(.20) + \tfrac{10}{100}(.30)$$

$$= .01 + .04 + .03 = \mathbf{0.08}.$$

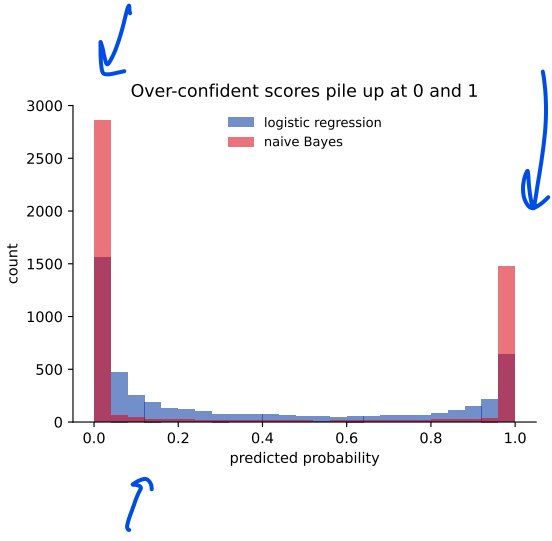
The two high-score bins are **over-confident** (say 0.7/0.9, only 0.5/0.6 actually bad) and drive almost all the error. *ECE is just a weighted average of the reliability diagram's vertical gaps.*

Over-confidence, seen another way

Same two models, histogram of their predicted probabilities.

- **Over-confident** models pile scores near 0 and 1 — they rarely say "maybe".
- Calibrated models spread mass across $[0, 1]$, matching real uncertainty.

Piling at the extremes is the fingerprint of miscalibration — and exactly what breaks any decision that reads $\hat{p}$ as a chance.



Calibration $\neq$ ranking

A model can order batches perfectly yet report wrong probabilities. AUC won't catch it — and any cost decision that uses $\hat{p}$ will be off.

Predict-first: perfect ranking, honest probabilities?

A cheese detector ranks **perfectly** — every bad batch scores above every clean one, so **ROC AUC** $= 1.0$. Now **halve every predicted probability** ($\hat{p} \to \hat{p}/2$).

Predict: what happens to the AUC? to the calibration?

- **AUC: unchanged** ($= 1.0$). Halving preserves the *order*, and AUC sees only order.
- **Calibration: wrecked.** Batches now scored 0.45 are still bad 90% of the time — the numbers stopped meaning anything.

Ranking and calibration are different properties. A model can order perfectly and still lie about probabilities — and AUC will never catch it.

Calibration is separate from discrimination

- **Discrimination** (ROC AUC): does the score *order* positives above negatives? Cares only about ranking.
- **Calibration**: are the score *values* true frequencies?

As the last slide showed, a model can reach **AUC** $= 1$ (perfect ranking) and still be badly miscalibrated — ranking and probability values are different properties.

Why it matters. The cost-optimal threshold $c^* = \frac{C_{FP}}{C_{FN}+C_{FP}}$ and *any* expected-value decision assume $\hat{p}$ is a true probability. Reporting "90% risk" to the factory (or a doctor) only means something if it is calibrated.

When calibration matters

The deciding question: does your decision read the value of $\hat{p}$ as a chance? If yes, calibrate.

- **You report the probability to someone who acts on the number** — "30% chance malignant" to a doctor, "90% bad" to the factory, "5% default" to a lender. A wrong 0.9 is a wrong decision.
- **Expected-value / cost decisions.** The cost-optimal threshold $c^* = \frac{C_{FP}}{C_{FN}+C_{FP}}$ (L13) plugs $\hat{p}$ into an expected-cost calculation — feed it garbage probabilities and c^* is wrong.
- **You combine probabilities** — averaging or ensembling several models, multiplying independent ones, or feeding $\hat{p}$ into a downstream probabilistic model. Only calibrated numbers combine correctly.

Rule: **if the number itself is consumed**, it has to be true.

When you can skip it

If you only ever compare scores — never read one as a chance — calibration changes nothing. A monotonic distortion leaves the order intact.

- **You only need the ranking:** ROC-AUC, PR ranking, top-k% / lift, "flag the worst 100 batches" — all depend on *order*, which miscalibration doesn't touch.
- **You tune the threshold directly** on your target metric (F1, cost) on a validation set — the threshold **absorbs** the miscalibration, so $\hat{p}$'s exact value never matters.

Even then, calibrated probabilities can still help *interpretability and monitoring* — they just won't change these particular decisions.

Golden rule (whenever you calibrate): fit on data the base model never saw, and don't re-tune the threshold on that same split — both bias the result optimistically.

Who is miscalibrated?

Usually fine

- **Logistic regression** — it minimizes log-loss, a *proper scoring rule* (one minimized only by reporting the true probabilities, so honesty is rewarded).

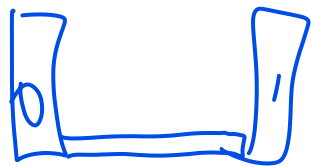
Over-confident (scores too extreme)

- Naive Bayes (independence assumption)
- Boosted trees, deep neural nets
- SVMs (margins, not probabilities)

Under-confident (scores too timid)

- Random forests (averaging pulls toward 0.5)

Watch out: `class_weight` and resampling (for imbalance) also distort probabilities — they change the base rate the model trains on, so re-calibrate afterwards.



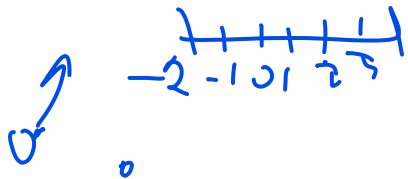
Measuring calibration

- **Reliability diagram** — the picture (qualitative).
- **ECE** (expected calibration error): bin predictions, average |observed − predicted| weighted by bin size. The headline number; 0 is perfect. *Caveat: depends on the binning.*
- **Brier score** $= \frac{1}{n}\sum(\hat{p}_i - y_i)^2$ — a proper scoring rule, but it *mixes* calibration and ranking, so a sharp model can score well while miscalibrated.
- **Log-loss** — also proper; punishes confident mistakes hardest.

For a *pure* calibration number use **ECE** (or read the reliability diagram); Brier and log-loss are good overall scores but conflate calibration with discrimination.

Fixing calibration

Don't retrain — learn a small post-hoc map from the model's raw scores to true probabilities, fit on held-out data.



Post-hoc calibration: the common idea

Don't retrain. Freeze the fitted model, then learn a one-dimensional map

$$g: \text{raw score } s \longmapsto \text{calibrated probability } \hat{p}$$

from a **held-out calibration set** of (score, label) pairs — data the base model never trained on.

- Two standard choices, differing only in how flexible g is: **Platt scaling** (a rigid sigmoid) and **isotonic regression** (a free monotonic curve).
- Both are **monotonic** maps, so they never reorder the scores — the **ranking (and AUC) is unchanged**; only the *values* move onto the diagonal.

Calibration is a cheap *post-processing* step: one extra fit on a held-out split, and the model itself is untouched.

Platt scaling: fit a sigmoid on the scores

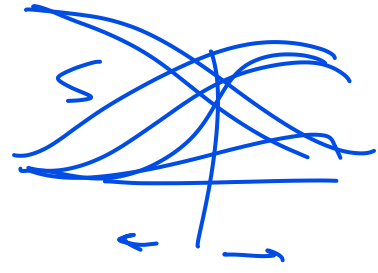
Push the raw score s through a **logistic sigmoid**:

$$\hat{p} = \sigma(as + b) = \frac{1}{1 + e^{-(as+b)}}.$$

- Fit just **two numbers** a, b by minimizing log-loss on the held-out (score, label) pairs — literally a 1-feature logistic regression *on the scores*.
- **Data-thrifty:** two parameters work with only a few hundred calibration points.
- **Assumption:** the distortion is a smooth **S-shape**. If the real miscalibration isn't sigmoidal, Platt can't remove it.

Originally **Platt (1999)**, to turn SVM margins into probabilities.

Neural nets: temperature scaling is the 1-parameter version $\hat{p} = \sigma(z/T)$ (fit only T on the logits z) — the standard fix (Guo et al., 2017).



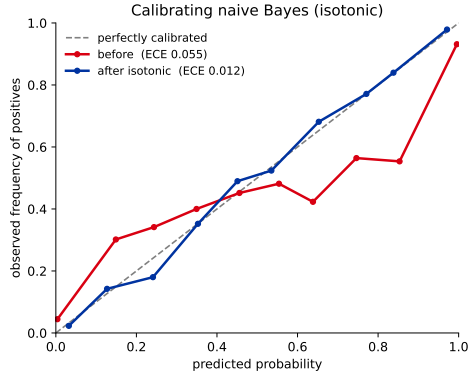
Isotonic regression: a free monotonic staircase

On what: the held-out pairs (s_i, y_i) — raw score and true label $y_i \in \{0, 1\}$ (same data as Platt).

How: fit the **non-decreasing** map m that matches those 0/1 labels in least squares:

$$m = \underset{g \text{ non-decr.}}{\arg\min} \sum_i \big(g(s_i) - y_i\big)^2$$

- **Pool-adjacent-violators:** sort by score; wherever the fit would *drop*, **pool** those points into their **average label** (the local positive rate); repeat until monotone $\Rightarrow$ a piecewise-constant **staircase**.



Snaps naive Bayes onto the diagonal: ECE $0.055 \to 0.012$.

No shape assumption $\Rightarrow$ fixes any monotonic distortion, but **needs more data** ($\gtrsim$ a few thousand) or the steps overfit noise.

Calibration in sklearn

```python
from sklearn.calibration import CalibratedClassifierCV

# wrap any classifier; calibrate by cross-validation on the training data
cal = CalibratedClassifierCV(
    GaussianNB(),
    method="isotonic",      # or "sigmoid" (Platt) for small data
    cv=5,
)
cal.fit(X_train, y_train)
cal.predict_proba(X_test)[:, 1]     # now-calibrated probabilities
```

- `method="sigmoid"` (Platt) for small data; `"isotonic"` for larger ($\gtrsim$ a few thousand).
- Calibrate on data the base model did **not** train on — `cv` handles this.
- `CalibrationDisplay.from_estimator(...)` draws the reliability diagram.

Recap

- **Calibration** = predicted probabilities match observed frequencies (read it off the **reliability diagram**).
- It is **separate from ranking**: high AUC does not imply good probabilities.
- Measure with **ECE** (pure calibration) or Brier / log-loss (overall, proper scores).
- Logistic regression is usually calibrated; naive Bayes, boosted trees and nets are over-confident; random forests under-confident; imbalance fixes distort probabilities.
- Fix post-hoc with **Platt** (sigmoid, small data) or **isotonic** (flexible, more data) via `CalibratedClassifierCV` — always on held-out data.

Next: imbalanced learning — class weights and resampling when the positive class is rare (and why they then need re-calibration).